Formation Kubernetes

Session 1 — De Docker à Kubernetes

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session

Pourquoi Kubernetes ?

Les limites de Docker seul, les défis de l'orchestration

15 min

Architecture Kubernetes

Control Plane, Worker Nodes, le modèle déclaratif

20 min

Les objets fondamentaux

Pod, Deployment, Service, Namespace

15 min

Travaux pratiques

Premier déploiement sur un cluster kind

1h05

Le problème

Docker seul ne suffit pas en production

Self-healing

Un container crashe.
Qui le redémarre ?

Scaling

Le trafic explose.
Comment ajouter des instances ?

Discovery

20 services, 50 containers.
Comment se trouvent-ils ?

Rolling updates

Nouvelle version.
Comment déployer sans downtime ?

Kubernetes résout tous ces problèmes.

Kubernetes (K8s)

Un orchestrateur de containers

Open-source, créé par Google en 2014, maintenu par la CNCF.

Vous lui décrivez l'état désiré de votre infrastructure, et il s'assure que la réalité corresponde à cet état.

Philosophie déclarative :
"Je veux 3 réplicas de mon API"
vs impératif :
"Lance un container, puis un autre, puis un autre"

- Déploiement automatisé
- Self-healing
- Scaling horizontal
- Service discovery
- Rollbacks automatiques
- Gestion des secrets

Architecture — Control Plane

Le cerveau du cluster

API Server

Point d'entrée unique. Tous les composants communiquent via lui.

etcd

Base de données clé-valeur. Stocke tout l'état du cluster.

Scheduler

Décide sur quel node placer chaque pod.

Controller Mgr

Boucle de réconciliation. Assure desired state = actual state.

Architecture — Worker Nodes

Les machines qui exécutent vos applications

kubelet

Agent sur chaque node. Reçoit les ordres de l'API Server et s'assure que les containers tournent.

kube-proxy

Gère le réseau. Redirige le trafic vers le bon pod grâce aux rules iptables/IPVS.

Container Runtime

Fait tourner les containers. containerd (standard), CRI-O, ou autre runtime compatible CRI.

Déclaratif vs Impératif

Impératif

"Fais ceci, puis cela"

```
$ docker run -d nginx
$ docker run -d nginx
$ docker run -d nginx

# Un crash ? Il faut le détecter
# et relancer manuellement...
```

Déclaratif

"Je veux cet état"

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3

# K8s maintient toujours 3 réplicas
# Crash ? Relance automatique.
```

Kubernetes réconcilie en permanence l'état désiré avec l'état réel.

Les objets fondamentaux

Pod

La plus petite unité déployable dans K8s.

Un pod = 1 ou plusieurs containers qui partagent le même réseau et stockage.

Chaque pod a sa propre adresse IP.

On ne crée presque jamais un pod directement — on utilise un Deployment.

Deployment

Gère les pods : combien de réplicas, rolling updates, rollbacks. C'est l'objet qu'on utilise 90% du temps.

Service

Expose les pods au réseau. Fournit une IP stable et un nom DNS. Types : ClusterIP, NodePort, LoadBalancer.

+ Namespace : isolation logique | + ConfigMap : configuration externalisée

kubectl — le couteau suisse

La CLI pour interagir avec votre cluster Kubernetes

`kubectl get pods`	Lister les pods en cours
`kubectl get pods -o wide`	Avec plus de détails (IP, node)
`kubectl describe pod <name>`	Détails complets d'un pod
`kubectl logs <pod>`	Voir les logs d'un container
`kubectl exec -it <pod> -- bash`	Ouvrir un shell dans un pod
`kubectl apply -f deploy.yaml`	Appliquer un fichier YAML
`kubectl delete pod <name>`	Supprimer un pod
`kubectl port-forward svc/api 8080:80`	Tunnel local vers un service

Astuce : kubectl est trop long ? Ajoutez alias k=kubectl dans votre .bashrc

TRAVAUX PRATIQUES

Premier déploiement sur Kubernetes

1. Installer kind et créer un cluster local
2. Explorer le cluster avec kubectl
3. Déployer un pod Nginx via YAML
4. Créer un Deployment avec 2 réplicas
5. Tuer un pod, observer la recréation
6. Modifier l'image, voir le rolling update

Durée : 1h05 | Outil : kind (Kubernetes in Docker) | Repo : github.com/reflektlab/k8s-training

Avant la prochaine session

Mini-défi (30 min)

Déployer votre propre image Docker sur le cluster kind :

1. Créer un Dockerfile pour une app simple (hello world en Go, Node, ou Python)
2. Builder l'image et la charger dans kind : kind load docker-image myapp:v1
3. Créer un Deployment YAML qui utilise votre image

Session 2 : Deployments, Services et le modèle réseau

Rolling updates, rollbacks, Service discovery DNS, communication inter-services